

Finding Minimum Dominating Set

Using Python Program in Certain Graphs

R. Manimegalai^{1||}, J. Anitha^{2||}, B.K. Danush Pranav^{3||}, S. Karthikeyan^{4||}

1,3,4 Department of Computer Science and Engineering, PSG Institute of Technology

and Applied Research, Coimbatore , India

¹Email: drmm@psgitech.ac.in

² Rajalakshmi Institute of Technology, Chennai, India

²Email: anitha.j@ritchennai.edu.in

ABSTRACT

This paper explores the Minimum Dominating Set (MDS) problem, focusing on developing and applying an algorithm suited to different graph structures. The algorithm is tested on various graphs, including path, ladder, cycle, comb, binary trees, and sibling trees. Through extensive testing, the paper evaluates the algorithm's performance in terms of solution quality, runtime efficiency, and scalability across diverse graph types. The findings provide valuable insights into algorithmic solutions for practical graph-based scenarios. Furthermore,

the algorithm exhibits potential applications in fields such as facility location, surveillance, and network design, demonstrating its versatility and relevance in real-world contexts.

Keywords : Dominating set; Facility location problem; Python program; Greedy algorithm; Algorithm optimization;

1 Introduction

The Minimum Dominating Set (MDS) in graph is a subset S of vertices within a graph such that every vertex in the graph is either included in the set or directly connected to the set of vertices. The cardinality of the minimum dominating set represents the minimum number of vertices that are required to achieve the condition of domination over the graph.

The minimum dominating set (MDS) problem plays a crucial role in the domain of facility location, where the objective is to position the fewest number of facility to serve efficiently for set of demand points. Minimum Dominating Set problem (MDSP) also applied in various domain like network monitoring, social networking etc.

It is an optimization problem in computer science and it is Np -hard . Identification of MDS is important as it have various application, starting from designing an efficient communicational network to reduce the spread of diseases in a population. Other than facility location and new monitoring, nowadays communication is growing exponentially and they play an important role in our daily life. They influence all the politics and economics. We communicate more with our friends and family using social networks than the face-to-face meeting. This connection forms a network so we need to reach all the nodes in the network quickly and directly due to other constraints.

This is possible if all the nodes are connected by at least one single node who is connected directly to them. So, we need to select a set of nodes in the network which can enable us to directly contact with all the other nodes in the network. In this case our main goal is to find a set which has minimum number of such nodes. In Graph theory this is known as the minimum dominating set problem [2]. There are several methods to solve the Minimum Dominating Set problem. Most methods used to solve this problem using approximation algorithms and heuristic methods [1]. Over the decades many greedy algorithms have been developed, each of this approaches has its own strengths and limitation. Graphs with substantially large number of vertices present a computational challenge when we try to identify the Minimum Dominating Set.

Terminology and Basic Concepts:

Formally a Graph is a collection of vertices and edges. Consider that vertices are dots and the edges are used to connect the dots. In mathematics a graph is denoted as (G) and the finite set of vertices (V) and a set of edges (E) which are connected to each other to form a graph / network. We can take Königsberg Bridge Problem as example. This problem consists of two islands which has 7 bridges and we need to start at one place and need to travel through all the bridges and reach the starting point again and we can travel only once in a bridge.

However, this is realized to go through every bridge and reach the same starting point. But we can represent this problem as a graph where all island can be represented as vertices and the bridges are the edges which forms and undirected graph network between the two islands.

A *path graph* (or linear graph) is a graph whose vertices can be listed in the order $v_1, v_2, \dots, v_n$ such that the edges are $\{v_i, v_{i+1}\}$ where $i = 1, 2, \dots, n - 1$. A *cycle* $C_n, n \geq 3$ is consist of n vertices $v_1, v_2, \dots, v_n$ and edges $\{v_1, v_2\}, \{v_2, v_3\}, \{v_3, v_4\}$ and

$\{v_{n-1}, v_1\}$. . A *comb* is a graph P_n^+ obtained by joining a single pendant edge to each vertex of a path. A *complete graph* is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge and is denoted by K_n of order n .

The *star graph* S_n of order n , is a tree on n nodes with one node having vertex degree $n - 1$ and the other $n - 1$ having vertex degree 1. The *Cartesian product* $G \square H$ of graphs G and H is a graph such that:

- The vertex set of $G \square H$ is the Cartesian product $V(G) \times V(H)$; and
- Two vertices (u, v) and (u', v') are adjacent in $G \square H$ if and only if either
- $u = u'$ and v is adjacent to v' in H , or
- $v = v'$ and u is adjacent to u' in G .

The ladder graph can be obtained as the Cartesian product of two path graphs, one of which has only one edge: $L_{n,1} = P_n \times P_2$.

The *friendship graph* or *fan graph* F_n can be constructed by joining n copies of the cycle graph C_3 with a common vertex, which becomes a universal vertex for the graph. F_n has $2n + 1$ vertices and $3n$ edges

A binary tree is a hierarchical data structure where every node has at most two child node, which are referred as left and right child of a node.

A sibling tree is a hierarchical data structure which is similar to a binary tree, where each node can have more than two child nodes and these child nodes are all siblings of each other.

2 Literature Review

2.1 Heuristic Approaches for Minimum Dominating Set

Guan, L., Wang, H. [1] presented a heuristic approximation algorithm for the minimum dominating set problem in undirected graphs. Their Heuristic Approximation Algorithm for Minimum Dominating Set (HAAMDS) utilizes principles from rough set theory. This employed a heuristic approach involving stepwise accumulation of data and forward/backward search mechanisms to enhance solution accuracy. The algorithm's feasibility is demonstrated through experimentation on public datasets, showcasing its practical applicability.

2.2 Equivalence of Graph Properties

Mofidi, A. [2] dove deep into the equivalence of properties related to dominating graphs, median graphs, and the complement of minimal dominating sets in graphs devoid of isolated vertices. Through an exploration of graph theoretical methods and concepts from domination theory, median graphs, partial cubes, and other graph properties, the study established connections between the discussed conditions, contributing to a deeper understanding of these graph structures and their interrelationships.

2.3 Challenges in Online Algorithms for Dominating Set

Boyar, et al. [3] examined the challenges inherent in online algorithms for the dominating set problem. Their research highlighted the difficulties posed by making irreversible decisions without foreknowledge of future input and the need to maintain solutions for all input prefixes. Through competitive analysis, the study sheds light on the performance of online algorithms in dominating set problems, offering insights into algorithmic abilities and their efficiency.

2.4 Variants of Dominating Set Problems

Ching-Hao Liu et al. [4] focused on the Independent Dominating Set Problem (IDS) within graph theory. Their analysis covers various aspects of the IDS problem, including its NP completeness in cubic grid and cubic bipartite graphs, fixed parameter tractable algorithms, and applications in identifying influential individuals in positive information propagation in social networks. By addressing different facets of the IDS problem, the study provided a comprehensive understanding of its complexity and applications.

2.5 Optimization Models and Algorithms for Dominating Set

Ding, X et al. [6] proposed a fast and differentiable optimization model for finding the minimum dominating set in large-scale graphs. By converting the coordinated model into a differentiable problem using gradient descent, the approach accesses all vertices simultaneously with linear time complexity. Comparative analysis demonstrates its efficiency, offering a promising alternative for solving the minimum dominating set problem and similar NP-

hard problems. The study contributed to algorithmic advancements in graph theory, particularly in optimizing solutions for large-scale graph problems.

2.6 Identification and Location-Domination Problems

Foucaud et al. [5] discussed identification, location-domination, and metric dimension problems, particularly in permutation graphs. Their work aimed to locate vertices within a graph using a minimal subset of selected vertices, proving the NP completeness of associated decision problems, including interval graphs of diameter 2. By addressing these fundamental problems in graph theory, the study provides insights into the complexity of graph structures and their associated decision problems.

2.7 Extension of Edge Graph Problems

Katrin Casel et al. [7] extended several edge graph problems by examining their standard parameterized and approximation complexity. Their study provided insights into various edge graph problems and contributed to understanding the computational complexity and potential approximation strategies for these problems.

2.8 Communication-Efficient Decentralized Deep Learning

Fateme Fotouhi et al. [8] addressed communication overhead in decentralized deep learning (DDL) and proposed the Minimum Connected Dominating Set Model Aggregation (DSMA) algorithm. Utilizing the Minimum Connected Dominating Set concept from graph theory, DSMA efficiently managed information flow among agents in DDL, showcasing scalability and competitive performance across various datasets, model architectures, and agent numbers. The study contributed to addressing scalability challenges in DDL, offering an efficient solution for information aggregation in decentralized environments.

2.9 Minimum Dominating Set for Graphs with Minimum Degree Three

Misa Nakanishi [9] focused on the Minimum Dominating Set problem for graphs with a minimum degree of at least 3. The study proposed a scheme to identify certain vertices in the minimum dominating set by transforming the graph, contributing to understanding the complexity of the Minimum Dominating Set problem for cubic graphs. By addressing specific graph structures, the study provides insights into the complexity of dominating set problems in graphs with minimum degree constraints.

2.10 Efficient Local Search Algorithm for Minimum Positive Influence Dominating Set

Rui Sun et al. [10] addressed the Minimum Positive Influence Dominating Set (MPIDS) problem in graph theory, particularly its applications in social networks. Their proposed Fast PIDS algorithm utilizes reduction-based initialization and a novel vertex exchange strategy to improve performance, outperforming state-of-the-art MPIDS algorithms on various benchmarks. By addressing efficiency challenges in MPIDS algorithms, the study contributes to advancing algorithms for positive influence propagation in social networks.

2.11 Paired Domination for Simple Graphs

S Sangeetha and M Swarnamalya [11] investigated paired domination in simple graphs, presenting results in the context of AIP Conference Proceedings. Their study emphasized the computational challenges and potential applications of paired domination, contributing to the broader understanding of graph theory and its practical implementations in network analysis and optimization.

2.12 Locating-Dominating Sets in Oriented Graphs

Nicolas Bousquet et al [12] explored the concept of locating-dominating sets, extending it from traditional graphs to oriented graphs. Their research, published in a recent volume, addressed the complexities and unique properties of oriented graphs, offering new insights into the applications and theoretical underpinnings of locating-dominating sets.

2.13 Locating and Paired-Dominating Sets in Graphs

John McCoy and Michael A. Henning [13] analysed the properties and applications of locating and paired-dominating sets in graphs. Their comprehensive study provided foundational knowledge and advanced the theoretical framework necessary for understanding the role of these sets in various graph structures.

2.14 NP-Completeness in Graph Partitioning

Mikko Koivisto et al [14] examined the NP-completeness results for partitioning a graph into total dominating sets. Their findings, published in a prominent journal, highlighted the computational difficulties associated with this problem, offering valuable insights for researchers and practitioners working on graph partitioning and related optimization challenges.

2.15 Average Order of Dominating Sets

Iain Beaton and Jasin I. Brown [15] investigated the average order of dominating sets in graphs. Their research provided statistical insights and theoretical foundations for understanding the distribution and characteristics of

dominating sets across different types of graphs, contributing to the broader field of graph theory and its applications.

Proposed Methodology

Due to the computational complexity of finding an exact solution to the MDS problem in large graphs (an NP-hard problem), a heuristic or approximation algorithm is often used.

Here is a practical approach using a greedy approximation algorithm implemented in Python:

1. **Graph Representation:** Represent the graph using adjacency lists or matrices, which are common and efficient ways to handle graph data structures.
2. **Greedy Algorithm:** Implement a greedy approach that iteratively selects the node with the highest degree (i.e., the most connections) that is not yet dominated. This method aims to cover the most nodes with the fewest selections.
3. **Validation:** Ensure that the selected nodes form a valid dominating set, meaning every node in the graph is either in the dominating set or adjacent to a node in the set.
4. **Optimization:** Optionally, apply further optimization techniques like local search or metaheuristic algorithms to refine the solution and potentially reduce the size of the dominating set.

Applications

Other than facility location and new monitoring, nowadays communication is growing exponentially and they play an important role in our daily life. They influence all the politics and economics. We communicate more with our friends and family using social networks than the face-to-face meeting. This connection forms a network so we need to reach all the nodes in the network quickly and directly due to other constraints. This is possible if all the

nodes are connected by at least one single node who is connected directly to them. So, we need to select a set of nodes in the network which can enable us to directly contact with all the other nodes in the network. In this case our main goal is to find a set which has minimum number of such nodes. In Graph theory this is known as the minimum dominating set problem.

Engineering Applications of Minimum Dominating Set:

- 1 **Wireless Sensor Networks:** Optimizing sensor placement is crucial to ensure full coverage while minimizing the number of sensors used, reducing costs, and improving network efficiency.
- 2 **Network Security:** Identifying critical nodes for monitoring and deducting intruders is essential to maintaining network security.
- 3 **Social Network Analysis:** Finding influential figures who can effectively spread information is common in social network analysis. An MDS helps identify these key influencers.
- 4 **Resource Allocation:** An MDS ensures all necessary areas are served with minimal resources. For example, health care facilities, schools, libraries, police stations etc.
- 5 **Power Grids:** Making sure that substations are operational and secure with the fewest of resources is vital for operational efficiency and reliability.

3 Algorithm for Finding Minimum Dominating Set of a Graph

Input : let G be a graph with n vertices

Output : set of nodes which gives the dominance over all the nodes.

function is_domination_set(graph, nodes)

not_visited $\leftarrow$ set(graph.nodes())

for each node in nodes **do**

if node in not_visited **then**

remove node from not_visited

end if

not_visited $\leftarrow$ not_visited - set(graph[node])

end for

return not not_visited

end function

function min_dominating_set(graph)

node_list $\leftarrow$ list(graph.nodes())

all_set_count $\leftarrow 2^{\text{len}(\text{node_list})}$

for i from 0 to all_set_count **do**

sample_set $\leftarrow$ []

for j from 0 to len(node_list) **do**

if ((i >> j) & 1 == 1) **then**

```

        append node_list[j] to sample_set

    end if

end for

if is_domination_set(graph, sample_set) then

    return sample_set

end if

end for

end function


function visualize_graph(graph, dominating_set)

    pos ← nx.spring_layout(graph)

    nx.draw(graph, pos, with_labels=True, font_weight='bold')

    nx.draw_networkx_nodes(graph, pos, nodelist=dominating_set, node_color='r',
label='Dominating Set')

    plt.show()

end function

```

Cycle Graph

Input :

edges= {

1: [2,8], 2: [1,3], 3: [2,4],4: [3,5],5: [4,6], 6: [5,7], 7: [6,8],8: [1,7]

}

Output:

```
>>> minimum dominating set : [1, 3, 6]
```

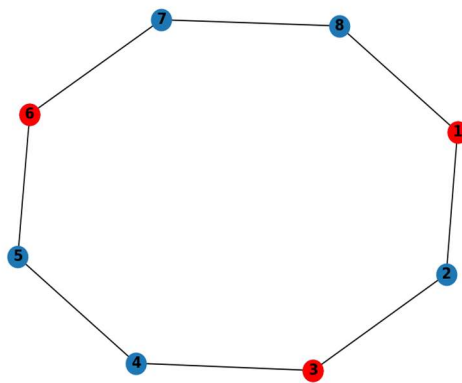


Figure 1. Red colored vertices indicates minimum dominating set of cycle graph

Linear Path Graph

edges = { 1: [2],2:[3],3:[4],4:[5],5 : [6],6:[7],7:[8] }

```
----- RESULT : D.V.V  
minimum dominating set : [1, 4, 7]
```

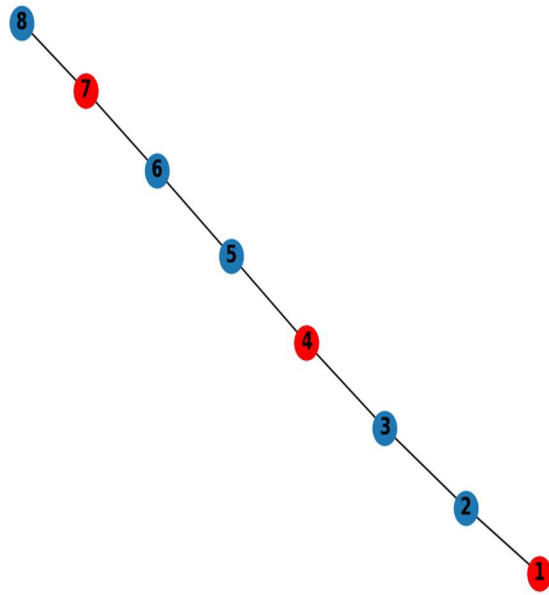


Figure 2. Red colored vertices indicates minimum dominating set of path graph

Star graph

edges3 = { 1: [2,3,4,5,6,7,8] }

minimum dominating set : [1]

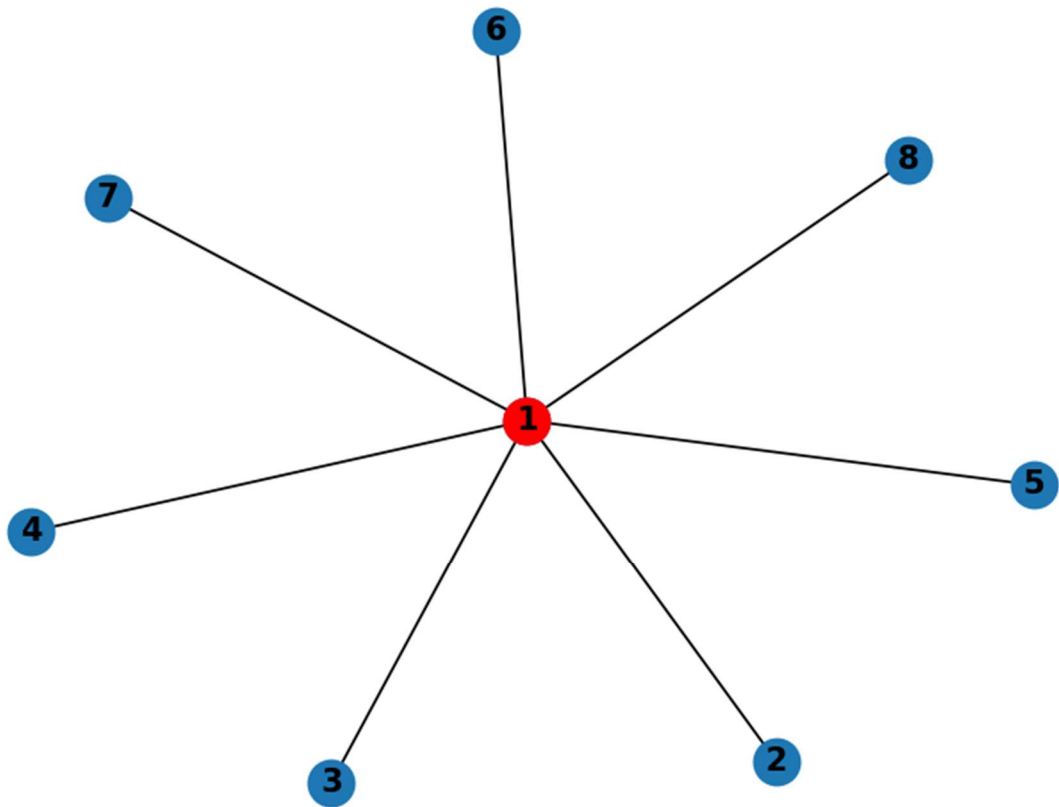


Figure 3. Red colored vertices indicates minimum dominating set of star graph

Fan graph

Input :

edges = { 1: [2,11],2: [1,11],3: [4,11],4: [11,3],5: [11,6],6: [5,11],7: [8,11],8: [7,11],9: [10,11],10: [9,11],11: [1,2,3,4,5,6,7,8,9,10]}

Output :

minimum dominating set : [11]

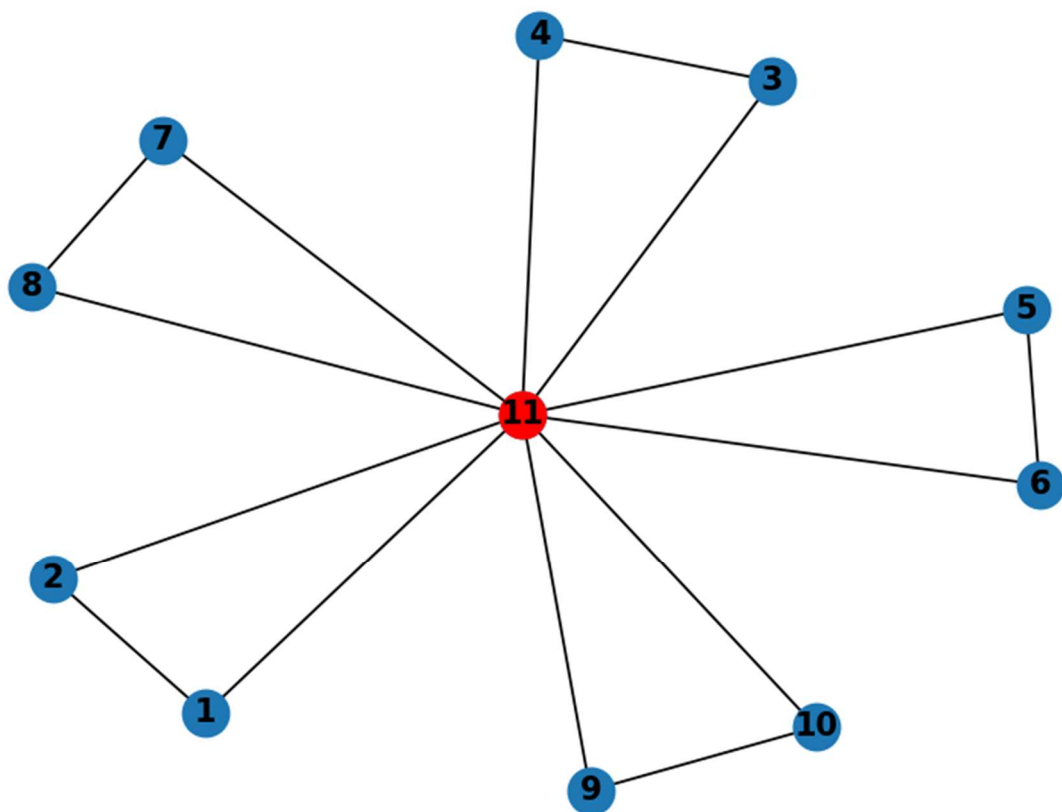


Figure 4. Red colored vertices indicates minimum dominating set of Fan graph

Ladder graph

Input :

edges = {

1: [2,6,7],2: [1,3,7,8],3: [2,4,8,9],4: [3,5,9,10],5: [4,10],6: [1,7],7: [1,2,6,8],8: [2,3,7,8],9:
[3,4,8,10],10: [9,5,4]
}

Output :

minimum dominating set : [4, 7]

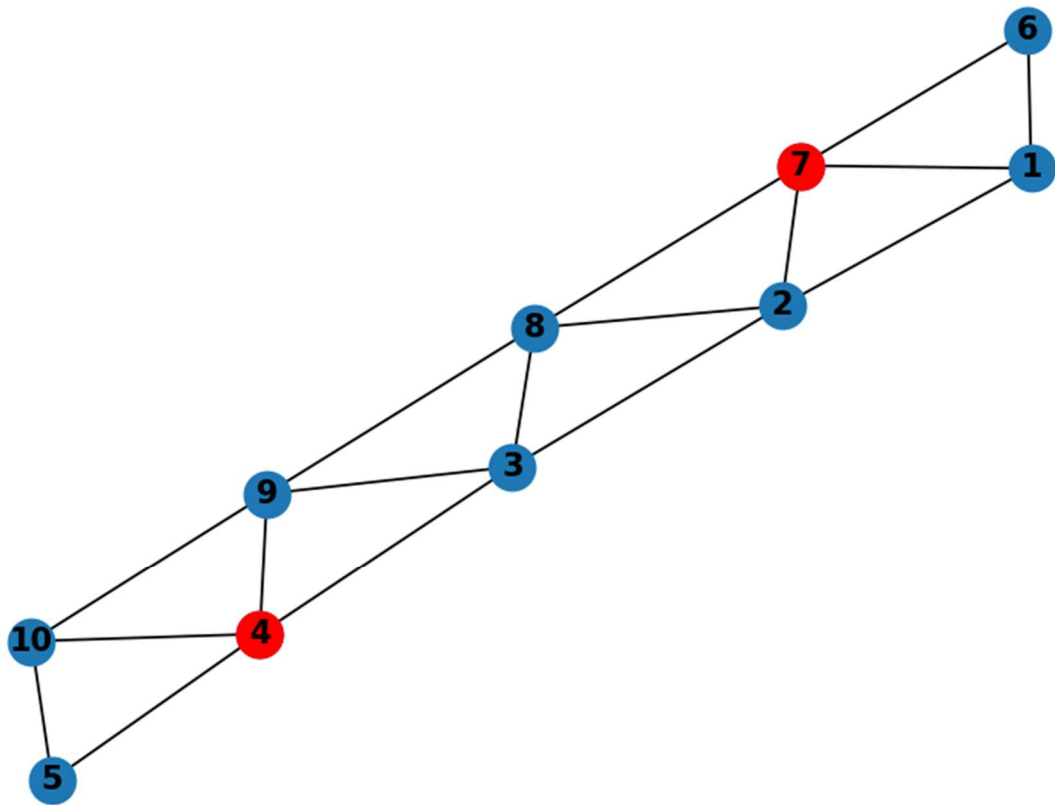


Figure 5. Red colored vertices indicates minimum dominating set of ladder graph

Binary tree

Input :

edges = { 1 : [2,3],2:[1,4,5],3:[1,6,7],4:[2,8,9],5 : [2,10,11],6:[3,12,13],7:[3,14,15],8:[4,16,17],9 :
[4,18,19],10:[5,20,21],11:[5,22,23],12:[6,24,25],13: [6,26,27],14:[7,28,29],15:[7,30,31],16:[8],17 :

[8],18:[9],19:[9],20:[10],21:[10],22:[11],23:[11],24:[12],25 :
 [12],26:[13],27:[13],28:[14],29:[14],30:[15],31:[15] }

Output:

minimum dominating set : [1, 8, 9, 10, 11, 12, 13, 14, 15]

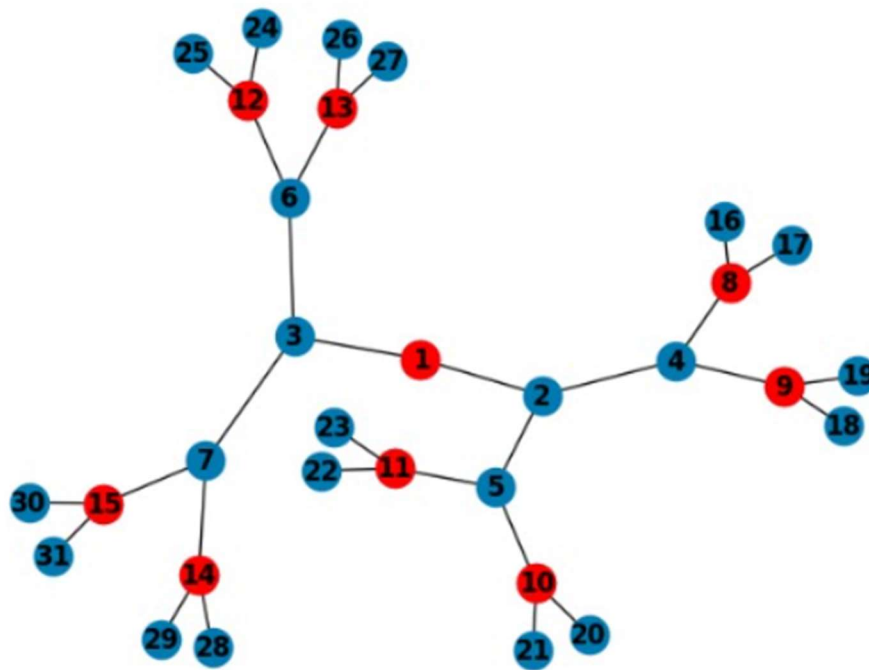


Figure 6. Red colored vertices indicates minimum dominating set of Binary Tree

Sibling Tree

Input :

edges7 = {

1 : [2,3],2:[4,5,1,3],3:[1,6,2,7],4:[2,5,8,9],5 : [2,4,10,11],6:[3,7,12,13],7:[3,6,14,15],8:[4,9,16,17],9 :
 [4,8,18,19],10:[5,11,20,21],11:[5,10,22,23],12:[6,13,24,25],13:
 [6,12,26,27],14:[7,15,28,29],15:[7,14,30,31],16:[8,17],17 : [8,16],18:[9,19],19:[9,18],20:[10,21],21:

```
[10,20],22:[11,23],23:[11,22],24:[12,25],25 :
[12,24],26:[13,27],27:[13,26],28:[14,29],29:[14,28],30:[15,31],31:[15,30]

}
```

Output :

```
minimum dominating set : [1, 9, 11, 13, 15, 16, 20, 24, 28]
```

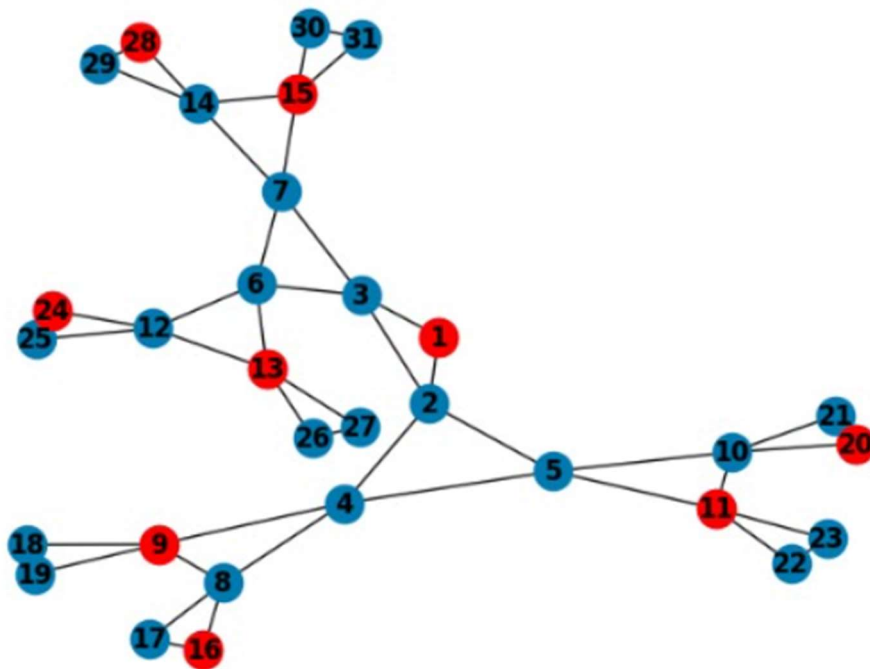


Figure 7. Red colored vertices indicates minimum dominating set of sibling tree

Comb Graph

Input :

```
edges = {

    1: [2,7],2: [1,3,8],3: [2,4,9],4: [3,5,10],5: [4,6,11],6: [5,12],7:[1],8:[2],9:
[3],10:[4],11:[5],12:[6]

}
```

Output :

minimum dominating set : [1, 2, 3, 4, 5, 6]

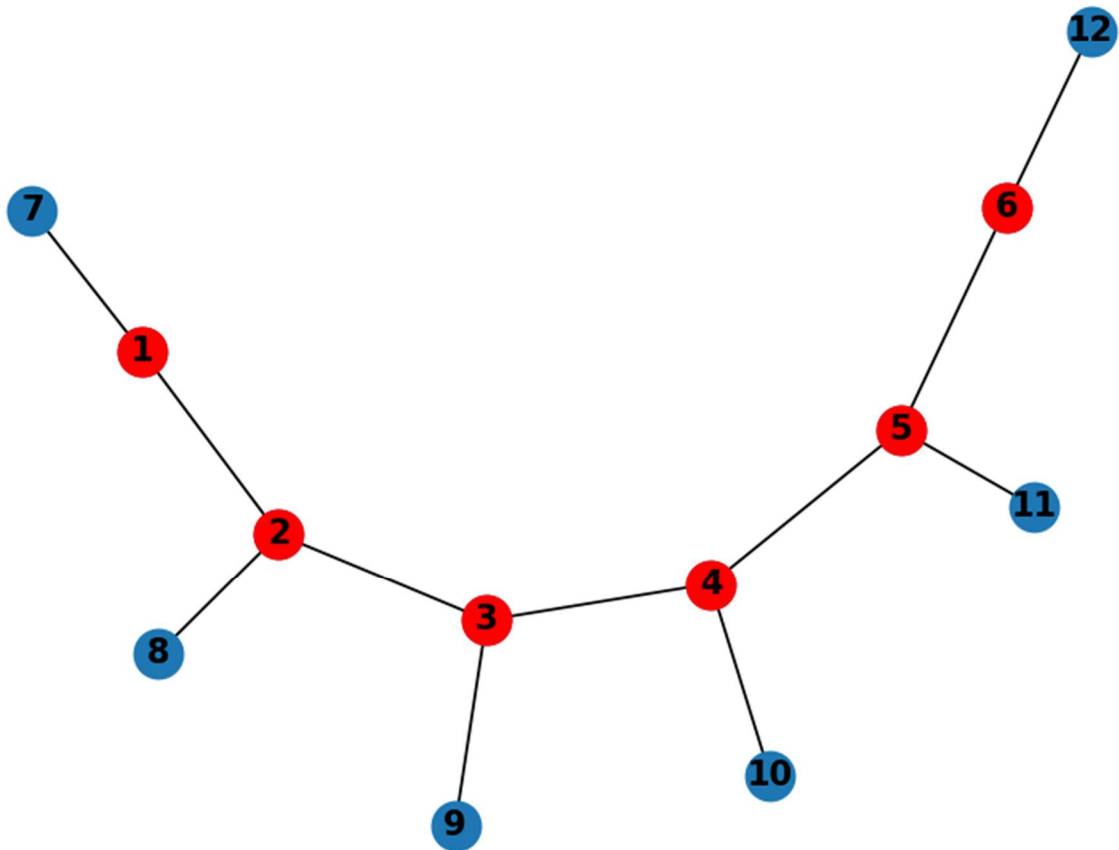


Figure 8. Red colored vertices indicates minimum dominating set of comb graph

The following Table 1 exhibits the minimum dominating set and the execution time for all the above graphs.

TYPE	NUMBER OF NODES	RUNTIME
Cycle Graph	8	0.000997s
Line Graph	8	0.004002s
Star Graph	8	0.002003s
Wheel Graph	11	0.028002s
Graph 5	10	0.018001s
Binary Tree	31	1.323242s
Sibling Tree	31	1868.682s
Ladder Graph	12	0.029000s
Comb Graph	12	0.045004s

Conclusions

In this paper, we have presented an algorithm designed to address the Minimum Dominating Set (MDS) problem across various graph structures including paths, cycles, ladder graphs, binary trees, and sibling trees. Our algorithm efficiently computes the minimum dominating set for each graph type, providing a practical solution to this fundamental problem in graph theory. By presenting the running time analysis for each network type, we offer insights into

the algorithm's computational efficiency and scalability. These findings not only contribute to the theoretical understanding of the MDS problem but also provide practical implications for its application in real-world scenarios. Moreover, the algorithm's ability to handle diverse graph structures underscores its versatility and utility in addressing a wide range of graph-based optimization challenges. Overall, this research lays the groundwork for further exploration and refinement of algorithms for the MDS problem, paving the way for advancements in graph theory and optimization techniques.

References :

- [1] Guan, L., Wang, H. A heuristic approximation algorithm of minimum dominating set based on rough set theory. *J Comb Optim* 44, 752–769 (2022)
- [2] Mofidi, A. On Dominating Graph of Graphs, Median Graphs, Partial Cubes and Complement of Minimal Dominating Sets. *Graphs and Combinatorics* 39, 104 (2023)
- [3] Boyar, J., Eidenbenz, S.J., Favrholt, L.M. *et al.* Online Dominating Set. *Algorithmica* 81, 1938–1964 (2019)
- [4] Ching-Hao Liu , Sheung-Hung Poon , Jin-Yong Lin *et al.* Independent dominating set problem revisited. [Volume 562](#), 11 January (2015), Pages 1-22
- [5] Foucaud, F, Mertzios, G.B., Naserasr, R. *et al.* Identification, Location-Domination and Metric Dimension on Interval and Permutation Graphs. II. Algorithms and Complexity. *Algorithmica* 78, 914–944 (2017)
- [6] Xiaojun Ding, Jianmei Chen, Jie Qiu *et al.* A fast and differentiable optimization model for finding the minimum dominating set in large-scale graphs. Volume 73, November (2023), 102146

- [7] Katrin Casel , Henning Fernau , Mehdi Khosravian Ghadikolaei, Jerome Monnot, Florian Sikora et al. Extension of some edge graph problems: Standard, parameterized and approximation complexity. Volume 340, 15 December (2023), Pages 183-201
- [8] Fateme Fotouhi, Aditya Balu , Zhanhong Jiang, Yasaman Esfandiari , Salman Jahani ,Soumik Sarkar et al. Dominating Set Model Aggregation for communication-efficient decentralized deep learning. Volume 171, March (2024), Pages 25-39
- [9] Misa Nakanishi. A note on vertices contained in the minimum dominating set of a graph with minimum degree three. Volume 956, 4 May (2023), 113831
- [10] Rui Sun , Jieyu Wu , Chenghou Jin , Yiyuan Wang , Wenbo Zhou , Minghao Yin et al . An efficient local search algorithm for minimum positive influence dominating set problem. Volume 154, June (2023), 106197
- [11] S Sangeetha and M Swarnamalya . Paired domination for some simple graphs,AIP Conference Proceedings, vol. 2112, AIP Publishing LLC, (2019), p. 020068
- [12] Nicolas Bousquet , Quentin Deschamps , Tuomo Lehtilä , Aline Parreau et al . Locating-dominating sets: From graphs to oriented graphs , Volume 346 , Issue 1 , January (2023), 113124
- [13] John McCoy, Michael A. Henning . Locating and paired-dominating sets in graphs , Volume 157 , Issue 15 , 6 August (2009), Pages 3268-3280
- [14] Mikko Koivisto , Petteri Laakkonen , Juho Lauri et al . NP-completeness results for partitioning a graph into total dominating sets , Volume 818, 24 May (2020), Pages 22-31
- [15] Iain Beaton, Jason I. Brown The average order of dominating sets of a graph , Volume 344, Issue 12 , December (2021), 112595

